

Sudoku-Tool

Lena Marie Ludwig, Jari Hartmann, Lisa Rappold

Technische Hochschule Ulm

Juli 2026

1 Einleitung

Dieses Projekt hat das Ziel, ein Sudoku-Tool zu entwickeln, das verschiedene Funktionen wie das Einlesen von Sudoku-Rätseln, die Validierung der Eingaben, die Lösung des Rätsels und die Ausgabe der Ergebnisse bietet.

Hierbei haben wir uns im ersten Schritt mit den Must-Have-Features beschäftigt. Das Programm kann ein Sudoku aus einer Textdatei einlesen und dieses auf Gültigkeit prüfen. Hierbei stellt es sicher, dass keine doppelten Zahlen in einer Zeile, Spalte oder einem Block vorkommen. Anschließend kann das Sudoku gelöst werden, wobei die Lösung auf der Konsole ausgegeben werden kann. Der Benutzer hat zu jedem Zeitpunkt die Möglichkeit, das Sudoku in einer Textdatei formatiert auszugeben.

In diesem Bericht stellen wir die Umsetzung und Funktionalität der einzelnen Funktionen vor.

2 read()

Eine `read()` Methode zum Einlesen von Sudoku-Rätseln ist essenziell für die Funktionalität des Solvers. Hierfür gibt der User sein Sudoku in die Textdatei `inputSudoku.txt` ein. Dann wird mittels `malloc()` Speicherplatz für eine Matrix `int**` reserviert. Anschließend wird versucht, die Textdatei im Lesemodus zu öffnen. Schlägt dies fehl, wenn beispielsweise die Datei im Verzeichnis fehlt, wird der Speicherplatz wieder freigegeben und das Programm bricht mit einer Fehlermeldung ab. Ist das Öffnen erfolgreich, liest die Methode mithilfe einer `while`-Schleife das Sudoku ein.

Hierbei werden kleine Fehler bei der Nutzereingabe ignoriert. Leerzeichen, senkrechte Striche, Bindestriche und leere Zeilen werden übersprungen.

Bereits bei dem Einlesen wird eine grundlegende Validierung vorgenommen. Die Methode wirft eine Fehlermeldung, wenn in einer Zeile, die nicht leer ist, mehr oder weniger als 9 Zahlen vorkommen. Außerdem werden nur Eingaben von 0 bis 9 akzeptiert. Die Zahl 0 steht hierbei für ein leeres Feld. Auch bei zu vielen oder zu wenigen Zeilen wird eine Fehlermeldung ausgegeben.

Handelt es sich bei dem eingegebenen Zeichen um eine Zahl zwischen 0 und 9, wird durch die Subtraktion des ASCII Wertes von 0 der korrekte Wert in der Matrix gespeichert. Zuletzt ruft die Funktion `printMatrix()`, um die eingelesene Matrix in der Konsole auszugeben.

3 validate()

Die Funktion `validate()` überprüft, ob das Sudoku gültig ist. Hierfür werden drei 9x9 Hilfs-Matrizen erstellt, die jeweils die Zeilen, Spalten und Blöcke des Sudokus repräsentieren. Hierfür wird `calloc` verwendet, um gleichzeitig die Werte auf 0 zu setzen.

Wird beispielsweise die erste Zeile des Sudokus überprüft, geht die Funktion mit einer verschachtelten `for`-Schleife jedes Element durch. Ist eine Zahl vorhanden, wird in der entsprechenden Zeile der Matrix `zeile` der Wert in Index `zahl-1` auf 1 gesetzt. Es repräsentiert also jede Spalte eine Zahl. Sobald diese vorhanden ist, wird die Spalte in der entsprechenden Zeile auf 1 gesetzt. Ist dieses Element in der Matrix schon 1, kommt diese Zahl doppelt vor und es wird eine Fehlermeldung ausgegeben.

Indem die Indizes der verschachtelten `for`-Schleifen getauscht werden, können die Spalten des Sudokus überprüft werden. Es wird mit der selben Logik vorgegangen. Hierbei repräsentiert jede Spalte in der Matrix `spalte` eine Spalte in dem Sudoku.

Das Validieren der Blöcke ist etwas komplexer. Hierfür werden zwei Hilfsarrays `BLOCKZEILE` und `BLOCKSPALTE` erstellt, die für jeden der neun Blöcke die Koordinaten der linken oberen Ecke speichern. Die Zeile wird ermittelt, indem der entsprechende Wert aus `BLOCKZEILE` mit `j/3` addiert wird. Da Nachkommastellen von Integern abgeschnitten werden, werden die entsprechenden Werte in `BLOCKZEILE` mit 0, 1 und 2 addiert, was die korrekte Zeile ergibt.

Die Spalte wird ermittelt, indem der entsprechende Wert aus `BLOCKSPALTE` mit `j%3` addiert wird. Hierdurch wird `BLOCKZEILE` mit der Sequenz 0, 1, 2 addiert, was die korrekte Spalte ergibt.

Hierbei wird jeder Block in der Matrix `block` in einer Zeile gespeichert. Auch hier repräsentiert jede Spalte eine Zahl.

4 Methodenname()

To do: Jari

5 solve()

To do: Lena

6 output()

Diese Funktion hat das Ziel, das Sudoku formatiert in einer Textdatei zu speichern.

Hierfür öffnet sie zunächst die Zielfeld `outputSudoku.txt` im Schreibmodus. Schlägt dies fehl, wird eine Fehlermeldung ausgegeben. Für die Ausgabe der Matrix wird eine `for`-Schleife über insgesamt elf Durchläufe genutzt, wobei neun für die Zeilen des Sudokus stehen und zwei für die Trennstriche zwischen den 3x3 Blöcken. Hierbei gibt es die Variable `verschiebung`, die dafür sorgt, dass trotz der Trennstriche die korrekten Zeilen der Matrix ausgegeben werden. Das erfolgt, indem die Variable jeweils bei dem Einfügen der horizontalen Trennstrichen hochgezählt wird. Dann wird jeweils die Matrix in der Zeile `i-verschiebung` ausgegeben. Die Trennstriche

werden bei den Durchläufen 3 und 7 ausgegeben. Die vertikalen Abtrennungen der 3x3 Blöcke werden manuell durch senkrechte Striche in der Ausgabe eingefügt. Nach der Ausgabe wird die Datei wieder geschlossen.